



CLASS TEST

COMS3005A – Advanced Analysis of Algorithms

Semester 1 2023

54 Marks (there are actually 55 marks – I added one)

Duration 90 minutes

This paper consists of 14 pages.

Instructions

1. Make sure you write your name and student number on the paper.
2. Answer on the question paper.
3. Answer all questions.
4. The mark for each question is given in brackets next to each question.
5. Read each question carefully before answering: always make sure you provide an answer to what is being asked.

Surname: _____

First name: _____

Student number: _____

Question 1

Computability

[6]

- (a) *Turing Machines (TMs)* are more powerful than *Pushdown Automata (PDAs)* and *PDAs* are more powerful than *Finite Automata (FAs)*.

Explain why this is the case.

(3)

Solution: The focus of the question is one what makes one machine more powerful than the other – this relates mainly to memory.

FAs have no memory ✓

PDAs have a stack and so have limited memory ✓

TMs have essentially unlimited memory – can write to the tape. ✓

- (b) Explain why the *Universal Turing Machine (UTM)* is an important mathematical model in computability theory.

(3)

Solution:

- It can execute any other TM.
- Any function that can be executed on a TM is computable
- Therefore any function that can be executed on a UTM is computable
- Can be used as a basis for decideability proofs – e.g. the *Halting Problem*.

✓ for each reasonable answer up to a maximum of 3

Question 2

Complexity

[12]

- (a) Suppose you analyse the runtime of a given algorithm and you come up with the complexity function below:

$$f(n) = 13n^2 + 15n + 345$$

What can you say about the order of complexity of the algorithm?

Justify your answers.

(4)

Solution:

- It is in $O(n^2)$ – can find c and n_0 to ensure $f(n) \leq cn^2, \forall n > n_0$.
- It is in $O(n^k) \forall k > 2$ – can use the same constants.
- It is in $\Omega(n^2)$ – choose $c = 10$ and $n_0 = 1$ then $0 \leq 10n^2 \leq 13n^2 + 15n + 345, \forall n > n_0$
- It is in $\Theta(n^2)$ because it is in $O(n^2)$ and in $\Omega(n^2)$
- and more possibilities...

✓ each, half a mark for a statement, and half a mark for a reason/justification.

Also for other correct comments.

Maximum of 4 marks.

- (b) Consider the algorithm given below which will return the smallest number in the list of numbers.

```

00  Algorithm myMin(myList, n)
      Input:  myList, n where myList is an array with n entries
              (indexed 0...n-1)
      Output:  myList[n-1] which contains the smallest number in the list
01      i ← n - 1
02      For j from 0 to i - 1
03          If myList[j] < myList[j + 1]
04              Then
05                  Swop(myList[j], myList[j + 1])
06      Return myList[n - 1]
```

- i. Explain how the algorithm solves the problem. (1)

Solution: Moves smallest number to the end of the list by repeatedly comparing pairs of numbers starting at the beginning of the list.

- ii. If *swops* are considered to be the *basic operation* of this algorithm, when would the *best* and *worst* cases occur? (2)

Solution: best case – no swops – implies list was originally reverse sorted. ✓

worst case – always swop – implies that the smallest number was in the first position in the list, would also accept a comment that list was already sorted in ascending order. ✓

- iii. Determine the *average* case performance of the algorithm in terms of the number of *swops* made to find the smallest number in a list of length n .

State any assumptions that you make.

(5)

Solution: Assume that smallest number could be in any position in the list ✓ and that the probabilities are equal (i.e. $1/n$ for a list of length n) ✓

Then get $\frac{1}{n}((n-1) + (n-2) + (n-3) + \dots + 2 + 1)$ ✓

$((n-1)$ swops if in position 0, $(n-2)$ swops if in position 1, etc.)

$$= \frac{1}{n} \frac{n(n-1)}{2}$$

$$= \frac{(n-1)}{2} \quad \checkmark$$

Average case is in $O(n)$. ✓

Question 3

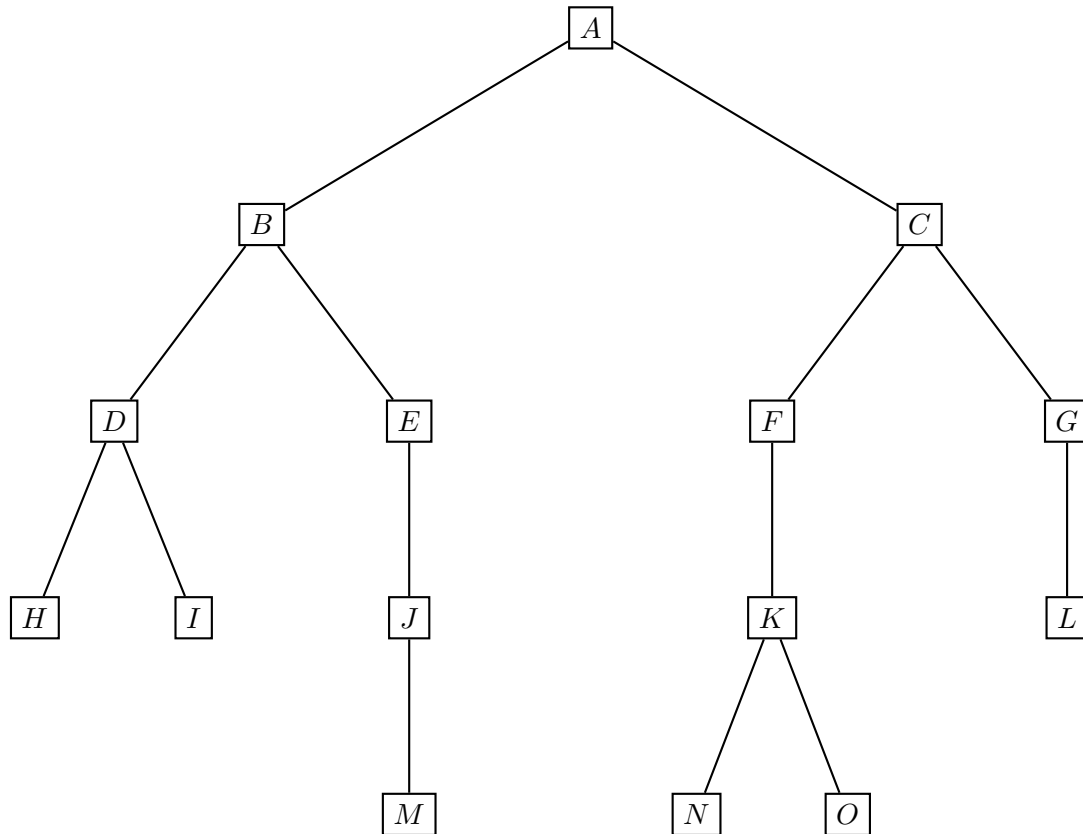
Uninformed Search

[4]

(a) Consider the tree shown below.

Show how **depth first search** would search the tree to find the node labeled *G*.

Be sure to show which node is being explored, which nodes have already been explored and which nodes are in the frontier.
(4)



Solution: Explore A – frontier is B,C

explore B – frontier is D, E, C

explore D – frontier is H, I, E, C

explore H – frontier is I, E, C

explore I – frontier is E, C

explore E – frontier is J, C

explore J – frontier is M, C

explore M – frontier is C

explore C – frontier is F, G

explore F – frontier is K, G

explore K – frontier of N, O, G

explore N – frontier is O, G

explore O – frontier is G

explore G – goal found.

✓ ✓ ✓ ✓ for perfectly correct solution

- 1 ✓ for each error

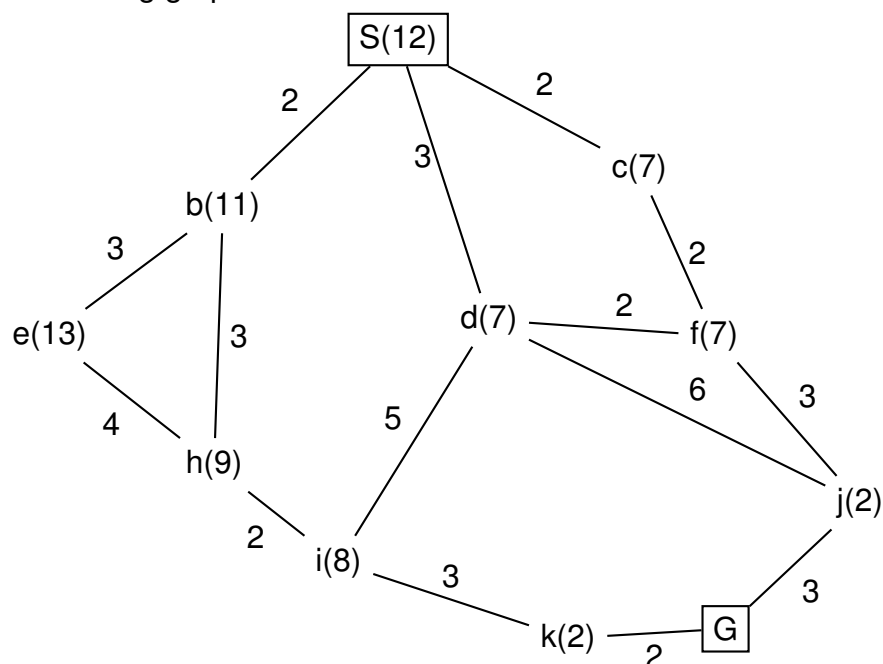
If the recursive DFS is being used then the *frontier* is essentially the unvisited nodes in the various recursive calls that have not yet been actually called. For the marks would then need to be a decent trace of the recursion.

Question 4

Informed Search

[17]

(a) Consider the following graph:



Each node shows the $\hat{h}$ value (estimate to the goal) in parentheses after the node name. Each edge shows the $\hat{g}$ value next to it.

Assume that an A* search is employed, that the start node is S , and the goal is G .

The first 2 steps (start node added to frontier, the expansion of that node, and the state of the frontier) are shown in the table below. The *node expanded* column shows the $\hat{g}$, $\hat{h}$, and $\hat{f}$ values in parentheses after the node name, and the frontier shows the $\hat{f}$ value in parentheses after the front-most node in the path has been explored):

Step	Node expanded	Frontier
1		[< $S(12)$ >]
2	$S(0, 12, 12)$	[< $c(9), S$ >, < $d(10), S$ >, < $b(13), S$ >]

Complete the table by showing the list of nodes in the order in which they are expanded (not just explored) and the state of the open queue when the nodes are expanded.

(8)

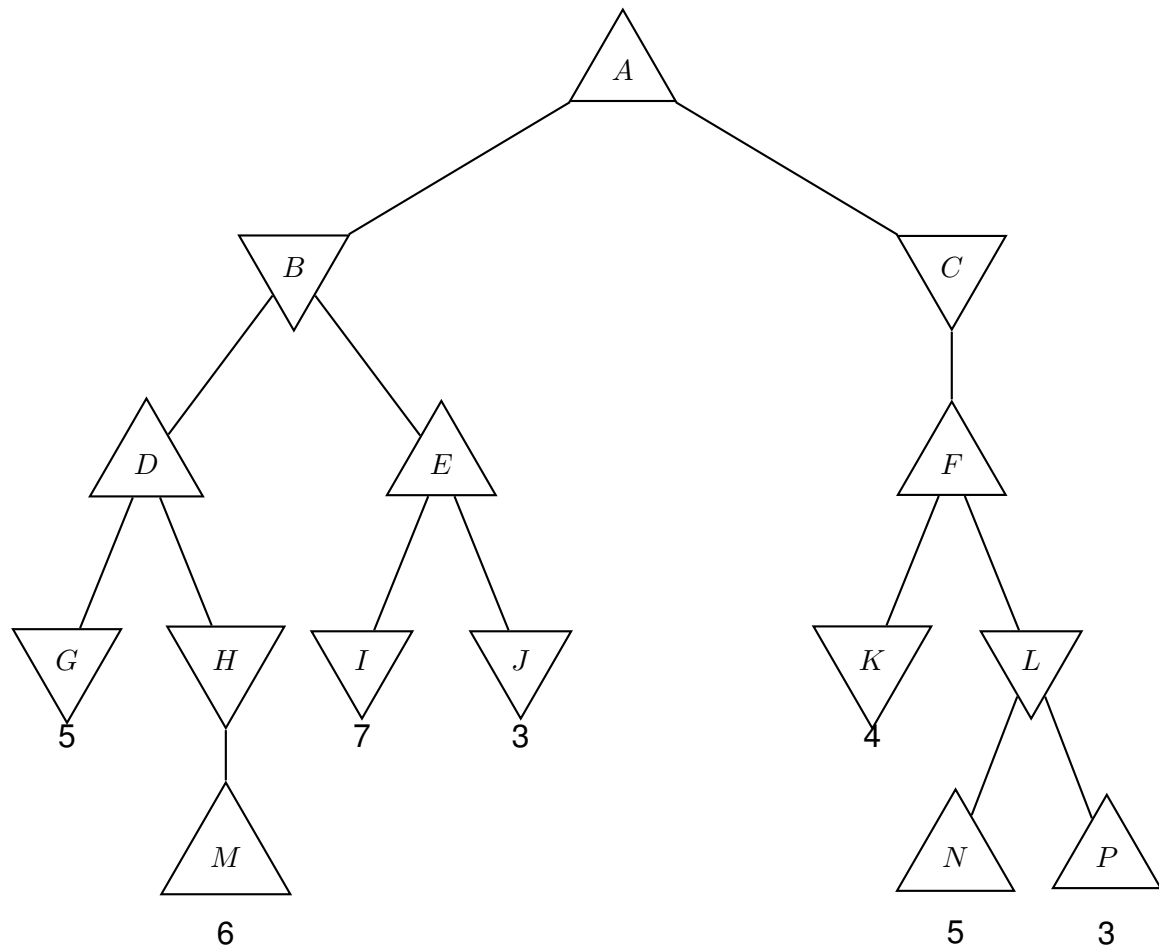
Solution:

Step	Node expanded	Frontier
1		[< S(12) >]
2	S(0, 12, 12)	[< c(9), S >, < d(10), S >, < b(13), S >]
3	c(2, 7, 9) ✓	[< d(10), S >, < f(11), c, S >, < b(13), S >] ✓
4	d(3, 7, 10) ✓	[< f(11), c, S >, < j(11), d, S >, < f(12), d, S >, < b(13), S >, < i(16), d, S >] ✓
5	f(4, 7, 11) ✓	[< j(9), f, c, S >, < j(11), d, S >, < f(12), d, S >, < b(13), S >, < i(16), d, S >] ✓
6	j(7, 2, 9) ✓	[< G(10), j, f, c, S >, < f(12), d, S >, < b(13), S >, < i(16), d, S >] ✓

Note that if there are the same values at any stage then the node that went into the frontier first is picked. Uses a priority queue as per Uniform Cost Search.

(b) Consider the game tree below and then answer the questions that follow.

Note: A is a *max* node and the static utility values for the leaf nodes are provided below each leaf node.



- i. Suppose a Minimax search is performed on the tree. Provide the min-max values for nodes A, B, C, D, E and F . (3)

Solution: $A = 6, B = 6, C = 4, D = 6, E = 7, F = 4$. 1/2 ✓ per correct answer

- ii. Suppose a Minimax search with Alpha/Beta pruning is done in a left-to-right fashion on the tree.
Determine if any subtrees are pruned in the process of searching the tree. State where the pruning happens and explain why the pruning of the tree at that point makes sense. (6)

Solution: I have worked this out laboriously to ensure I have the correct answers.

1. A: $\max - v = -\infty, \alpha = -\infty, \beta = \infty$ expand B
2. B: $\min - v = \infty, \alpha = -\infty, \beta = \infty$, expand D
3. D: $\max - v = -\infty, \alpha = -\infty, \beta = \infty$ expand G
4. G: $\min$ – return utility value, ie return 5
5. D: $\max - v = \cancel{\infty} 5$ v less than β , $\alpha = \cancel{\infty} 5, \beta = \infty$, Expand H ✓
6. H: $\min - v = \infty, \alpha = 5, \beta = \infty$, expand M ✓
7. M: $\max$ – return utility value, ie return 6 ✓
8. H: $\min - v = \infty, \alpha = 5, \beta = \infty, v = \cancel{\infty} 6$, v not less than α , $\beta = \cancel{\infty} 6$, Return $v = 6$.
9. D: $\max - v = \cancel{\infty} 6$ v less than β , $\alpha = \cancel{\infty} 6, \beta = \infty$, Return 6 ✓
10. B: $\min - v = \cancel{\infty} 6, \alpha = -\infty, \beta = \cancel{\infty} 6$, expand E ✓
11. E: $\max - v = -\infty, \alpha = -\infty, \beta = 6$ expand I ✓
12. I: $\min$ – return utility value, ie return 7 ✓
13. E: $\max - v = \cancel{\infty} 7$ $v \geq \beta$ so return 7, J is β cut.
This makes sense as no matter what J is B has a choice of 6 or at least 7 and will choose the 6.
Essentially ✓ ✓ ✓ for this step
14. B: $\min v = 6$ (min of 6 and the returned 7), $\alpha = -\infty, \beta = 6$.
Return 6 to A ✓
15. A: $\max - v = -\infty, \alpha = -\infty, \beta = \infty$
 $v = \text{MAX}(v, 6) = 6$
 $\alpha = \text{MAX}(\alpha, v) = 6$
 $v = \cancel{\infty} 6, \alpha = \cancel{\infty} 6, \beta = \infty$ ✓
Done with left subtree...
Expand C
16. C: $\min v = +\infty, \alpha = 6, \beta = \infty$, expand F.

17. F: $\max v = -\infty, \alpha = 6, \beta = \infty$, expand K

18. K: min, leaf, return utility value, i.e. 4.

19. F: $\max v = \cancel{-\infty} 4, \alpha = 6, \beta = \infty$, expand L

20. L: $\min v = +\infty, \alpha = 6, \beta = \infty$, Expand N

21. N: max, leaf return utility value = 5,

22. L: $\min v = \cancel{+\infty} 5, \alpha = 6, \beta = \infty, v$ less than α , return v , P is α cut

This makes sense as whatever happens in the left subtree L would be at least 5 but F would be at most 4 and therefore C would be 4 and A could get 6 by playing the left subtree of the original game tree.

Essentially ✓ ✓ ✓ for this step

Question 5

Classical Search

[6]

- (a) Suppose you are running the version of the linear search that handles the situation where the *key* may or may not be in the given input list.

In order to empirically determine the average case running time of the algorithm, you repeatedly generate a random number and search for this in a list of a given length. You record the runtime for each search and then average these times and record that time as the average runtime for a list of that input size.

Explain why this approach to determining the average runtime could give you a biased measurement of the average case. (3)

Solution: This came up in the lab.

If one randomly selects numbers then one cannot be sure the number is in the list. That means for such a run one would actually be measuring the worst case runtime for that input instance.

If this happens often in the testing then the average could be biased towards the worst case runtime. ✓ ✓ ✓

- (b) Suppose that you are running the bisection search algorithm on a list of numbers that contains a number of repeats of the key you are searching for.

Could this affect the number of comparisons made to find the key? Explain your answer. (3)

Solution: It is likely to cut down on the number of comparisons. ✓

This is because normally finding the key could involve bisecting the list until there is only one element in the list. In this case as you get closer to the repeats you could end up with a sublist where all the numbers are the key (repeats would obviously appear next to each other in the list as it is sorted) and so wherever you look in that sublist you will find the key. ✓ ✓

Question 6

Sorting

[10]

- (a) Trace the execution of *mergesort* on the list of numbers given below.

23, 45, 13, 19, 17, 34, 26, 55

(5)

Solution: This is largely mechanical.

Should demonstrate that they know how the algorithm works and how recursion works.

mergesort(23, 45, 13, 19, 17, 34, 26, 55)

mergesort(23, 45, 13, 19)

mergesort(23,45)

mergesort(23)

mergesort(45)

merge(23) and (45) in extra storage

mergesort(13,19)

mergesort(13)

mergesort(19)

merge(13) and (19) in extra storage

merge(23, 45, 13, 19) in extra storage returns (13, 19, 23, 45)

Detail on how the merging will actually occur would be nice but if it is missing then there will be no penalty.

mergesort(17, 34, 26, 55)

mergesort(17,34)

mergesort(17)

mergesort(34)

merge (17) and (34) in extra storage

mergesort(26,55)

mergesort(26)

mergesort(55)

merge (26) and (55) in extra storage

merge(17, 34, 26, 55) returns (17, 26, 34, 55)

merge (13, 19, 23, 45) and (17, 26, 34, 55) in extra storage

returns sorted list.

✓ ✓ for getting the recursive flow right ✓ ✓ ✓ for all steps correct including an indication of the use of extra storage and how the merge is handled in practice.

If no clear indication of recursive flow and no clear indication of extra storage then at most ✓ ✓ should be awarded.

- (b) Explain what would happen in the first call to the *quicksort* algorithm if the *pivot* is the smallest number in the list of numbers to be sorted.

(5)

Solution: Note in the algorithm that we studied the pivot is the last element in the list. (In lectures we discussed other ways to choose the pivot but those are not the algorithm we studied).

Here the question essentially says that the smallest number is in the pivot position

– the last position in the list.

The question did not mean – find the smallest number and make that the pivot. Besides being a bad strategy it would add extra cost to the algorithm at each stage of the recursion.

Recall that in a classical *divide and conquer algorithm* each recursive call has three parts

1. divide
2. apply to divided parts
3. combine solutions from parts

In quicksort, the partition is part one and is the important phase. Combining is trivial.

So the answer:

Left pointer will immediately find a number greater than the pivot – in the first position of the list ✓

Right pointer will traverse entire list looking for a number that is smaller than the pivot. ✓

Pivot and first element (where left pointer points) will be swapped ✓

quicksort will be called with empty list ✓ (to the left of where pivot element was moved to) and a list of length $n - 1$ ✓ (the second element to the last element in the list).

Total: 55

End of paper